

Datentyp `int` in Java

Thema	Beschreibung
Typ	<code>int</code> (primitive data type)
Größe	32 Bit (4 Byte)
Wertebereich	<code>-2.147.483.648</code> bis <code>2.147.483.647</code> (-2^{31} bis $2^{31} - 1$)
Standardwert	<code>0</code>
Wrapper-Klasse	<code>Integer</code> (<code>java.lang.Integer</code>)
Speicherung	Zweierkomplement-Darstellung
Explizite Umwandlung (Casting)	<code>(int) longwert</code> – Notwendig, wenn ein <code>long</code> -Wert auf <code>int</code> reduziert wird (Datenverlust möglich)
Implizite Umwandlung	<code>int</code> kann automatisch in <code>long</code> , <code>float</code> oder <code>double</code> konvertiert werden
Numerische Operationen	Addition (+), Subtraktion (-), Multiplikation (*), Division (/), Modulo (%)
Ganzzahlige Division	<code>int</code> -Division verwirft Nachkommastellen (<code>5 / 2</code> ergibt <code>2</code>)
Lösung für echte Division	<code>double result = 5 / 2.0; // Ergebnis: 2.5</code>
Vergleichsoperatoren	<code>==</code> , <code>!=</code> , <code><</code> , <code>></code> , <code><=</code> , <code>>=</code>
Bitweise Operatoren	<code>&</code> , <code> </code> , <code>^</code> , <code>~</code> , <code><<</code> , <code>>></code> , <code>>>></code>
Overflow & Underflow	<code>int</code> überläuft ohne Fehler (<code>int x = Integer.MAX_VALUE + 1; //</code> <code>ergibt -2.147.483.648</code>)
Lösung für Overflow-Prüfung	<code>Math.addExact(int a, int b)</code> , <code>Math.multiplyExact(int a, int b)</code> werfen <code>ArithmeticException</code>
Wrapper-Methoden	<code>Integer.parseInt(String s)</code> , <code>Integer.valueOf(String s)</code> , <code>Integer.toString(int i)</code>
Autoboxing & Unboxing	Automatische Umwandlung zwischen <code>int</code> und <code>Integer</code> (<code>Integer obj = 10; int x = obj;</code>)
Konstante Werte	<code>Integer.MIN_VALUE = -2_147_483_648</code> , <code>Integer.MAX_VALUE = 2_147_483_647</code>
Typischer Anwendungsfall	Standard-Datentyp für numerische Berechnungen, Zähler in Schleifen, IDs

